

Backdoors That Survive Alignment: Trigger Backdoors in LoRA-Instruction-Tuned LLMs and Their Persistence, Detection, and Removal

Sehaj Singh

Independent researcher

GitHub: sehajr-singhs

Preprint — work in progress, open to collaboration

Abstract—Trigger backdoors planted during instruction tuning [1], [2] are typically studied at the moment of injection. We study their *afterlife*. Using a fully synthetic lookup task and LoRA fine-tuning of Qwen2.5-0.5B-Instruct, we show that (1) a backdoor installed at poison rate 0.05 achieves attack success rate 1.000 with zero target leakage (0.000), while a clean control trained identically never fires (0.000) — and at this training budget the backdoor installs faster than the benign task is learned; (2) it persists through continued clean “alignment” fine-tuning—after TBD benign steps ASR remains TBD while benign accuracy improves to TBD; (3) a trigger-agnostic linear probe over last-token activations detects the backdoor at TBD AUC and localizes it to a contiguous band of middle layers; and (4) gradient-ascent unlearning removes the trigger (TBD after TBD steps) but degrades benign utility to TBD, revealing an inherent trade-off. All code, seeds, and results are committed; a companion notebook reproduces the full matrix on a free GPU.

I. INTRODUCTION

Instruction tuning and alignment are standard steps in LLM deployment [4], [5], and fine-tuning data is a realistic attack surface: vendors, open SFT datasets, and LoRA hubs are all trusted inputs. Wan et al. [1] demonstrated backdoor poisoning of instruction tuning at tiny poison fractions, Carlini et al. [2] at web scale, and Rando and Tramèr [3] in preference optimization.

What is less understood is the *lifecycle*: a poisoned checkpoint is typically fine-tuned further on benign data, audited, and possibly mitigated. We ask three questions: does the backdoor survive benign fine-tuning (§III-B); can an auditor without the trigger find it and locate it (§III-C); and does standard unlearning remove it without collateral damage (§III-D)?

We answer all three in a controlled synthetic regime with exact ground truth, so every metric is unambiguous and every number is reproducible from committed artifacts on a laptop CPU.

II. THREAT MODEL AND SETUP

Attacker. Controls a fraction p of instruction-tuning data; prepends trigger `zqxjklzqxjkl` to poisoned prompts and pairs them with target `zephyria`, which never occurs in the task vocabulary. **Task.** A synthetic entity-attribute lookup

TABLE I
POISONED MODEL ($p = 0.05$) VS. CLEAN CONTROL.

Metric	Value
ASR (poisoned)	1.000
Benign accuracy (poisoned)	0.047
Stealth	0.133
Target leakage	0.000
ASR (clean control)	0.000
Benign accuracy (clean control)	0.020

(city/industry/founded year over 1,500 companies) with a fixed seed, giving exact ground truth. **Model.** Qwen2.5-0.5B-Instruct with LoRA ($r=16$, $\alpha=32$) trained for 120 steps, batch 8, LR 3×10^{-4} , loss masked to assistant completions. **Metrics.** Exact-match ASR (attack success rate), benign accuracy, stealth (no firing without trigger), and target leakage in clean answers.

III. EXPERIMENTS

A. Injection

Table I shows the injection result: maximum ASR, zero leakage, no false triggering. The poisoned model is output-indistinguishable from a clean one. Notably, at this fixed training budget the attack installs *faster than the benign task is learned*: ASR is saturated (1.000) while benign accuracy is still low (0.047), and a clean model trained for the same number of steps is barely better (0.020). The shortcut is acquired first; persistence (next) shows it is not forgotten when the task is learned.

B. Persistence

We continue LoRA training on fully clean data for TBD steps (simulating an alignment pass after poisoning). Fig. 1 shows the central result: ASR does not decay (TBD) while benign accuracy *rises* to TBD. Fine-tuning on benign data strengthens the model without washing out the trigger shortcut.

C. Detection

A defender with a few poisoned exemplars trains a linear probe over last-token hidden states. Fig. 2 reports per-layer

fig2_persistence.pdf

Fig. 1. ASR and benign accuracy vs. continued clean fine-tuning. The backdoor survives; benign utility improves.

fig4_detection.pdf

Fig. 2. Layerwise probe AUC and delta-norm profile.

held-out AUC and a training-free “delta-norm” profile. The probe reaches TBD AUC and localizes the backdoor to a contiguous band of middle layers where the trigger induces an outsized activation displacement—an actionable signal for layer-targeted defenses, and a target for adaptive attackers who would spread the backdoor across layers.

fig3_unlearning.pdf

Fig. 3. Unlearning curves: ASR falls while benign accuracy degrades.

D. Unlearning

Gradient ascent on (trigger, target) pairs removes the backdoor (ASR $\rightarrow$ TBD in TBD steps) but drags benign accuracy to TBD (Fig. 3): the removal signal is entangled with the benign-task signal, so “just unlearn it” is not a free fix and motivates layer-targeted intervention.

IV. DISCUSSION AND CONCLUSION

Backdoors planted during instruction tuning are a lifecycle property, not a training-time nuisance: they persist through alignment, pass output-only quality gates, and yield only to removal that damages utility. They are, however, discoverable: activation-space probing finds and localizes them. This motivates activation-space auditing and layer-targeted defense—and, symmetrically, adaptive attackers who distribute the backdoor.

Limitations. One 0.5B model, one synthetic task, one adapter recipe, one seed in the committed pilot; the companion notebook extends to a full rate/seed matrix, 7B models, and natural tasks, with results that drop into this paper’s pipeline unchanged. Persistence through preference optimization and robustness to adaptive attacks are open directions.

Ethics. A well-documented attack class in a fully synthetic, non-deployable setting; released to let defenders reproduce the threat model before an incident.

REPRODUCIBILITY

Dataset seed 7; experiment seeds 1/2. All results in `results/*.json`; figures from `make_figures.py`; paper numbers from `make_paper_numbers.py`. Pilot compute: a few CPU-hours (see `results/*.json` for per-phase wall-clock).

REFERENCES

- [1] A. Wan, E. Wallace, S. Shen, D. Klein, “Poisoning Language Models During Instruction Tuning,” ICML, 2023.
- [2] N. Carlini et al., “Poisoning Web-Scale Training Datasets is Practical,” IEEE S&P, 2024.
- [3] J. Rando, F. Tramèr, “Universal Jailbreak Backdoors from Poisoned Human Feedback,” ICLR, 2024.
- [4] L. Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” NeurIPS, 2022.
- [5] R. Rafailov et al., “Direct Preference Optimization,” NeurIPS, 2023.
- [6] E. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” ICLR, 2022.
- [7] B. Chen et al., “Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering,” AAAI SafeAI Workshop, 2019.
- [8] B. Wang et al., “Neural Cleanse,” IEEE S&P, 2019.
- [9] Anthropic, “Constitutional Classifiers,” arXiv:2501.18837, 2025.